

Primitive Ring Cancellation Specification

Stage 7R Revision: Exact Finite GF(2) Solver with Explicit Memory Resources

mdstats

2026-07-18

Contents

Purpose and boundary	1
Physical support	1
Finite algebra	1
Public API	1
Memory preflight	2
Input constraints	2
Positive certificate verification	2
Result boundary	2
Complexity	3
Explicit non-responsibilities	3
References	3

Purpose and boundary

`primitive_ring_cancellation.py` is the low-level exact finite solver for modulo-two cancellation of translated primitive-ring supports.

Runtime/API target:

mdstats 0.19.19a0

This module consumes an explicit finite candidate set. It does not define a strength search domain, enumerate candidates, or publish a global strong-ring claim. Retaining explicit candidates in the transient low-level solve result is intentional; persistent strength records are handled by `ring_strength.py`.

Physical support

A `RingPlacement` is converted into a sorted tuple of exact physical lifted-edge instances:

```
@dataclass(frozen=True)
class RingPlacementSupport:
    placement: RingPlacement
    edge_instances: tuple[LiftedEdgeInstanceRef, ...]
```

Support identity includes the topology graph digest, stable `FrameworkEdgeKey`, and physical anchor shift. Quotient edge position alone is not a physical support identity.

Finite algebra

Given target vector b and candidate columns c_j over physical edge instances, solve

$$Cx = b \pmod{2}.$$

Each support vector is represented as a Python integer bitset. Gaussian elimination maintains a corresponding provenance bitset so a positive solution can be reconstructed exactly.

Public API

```
@dataclass(frozen=True)
class FiniteRingCancellationResources:
```

```

max_matrix_bits: int = ...
max_provenance_bits: int = ...
ring_placement_support(
    index: PrimitiveRingIndex,
    placement: RingPlacement,
) -> RingPlacementSupport
solve_finite_ring_cancellation(
    index: PrimitiveRingIndex,
    target_placement: RingPlacement,
    candidate_placements: Iterable[RingPlacement],
    *,
    resources: FiniteRingCancellationResources | None = None,
) -> FiniteRingCancellationResult

```

Result statuses:

```

DECOMPOSITION_FOUND
NOT_IN_SUPPLIED_SPAN

```

NOT_IN_SUPPLIED_SPAN means only that the target is absent from the explicitly supplied finite span.

Memory preflight

Let N be candidate count and E the number of represented physical edge instances. Before constructing the elimination basis, estimate

$$B_{\text{matrix}} = NE,$$

$$B_{\text{provenance}} = N \min(N, E).$$

If either estimate exceeds its declared limit, raise `PrimitiveRingCancellationResourceError` before the potentially large allocation. Higher-level strength classification converts this exception into `UNRESOLVED_TRUNCATED`.

These bounds are conservative representation guards, not mathematical search limits.

Input constraints

- The target and every candidate must be a `RingPlacement` from the same topology graph digest as the `PrimitiveRingIndex`.
- Duplicate exact candidate placements are rejected.
- Candidate ordering is canonicalized deterministically.
- Every referenced ring key must exist in the source index.
- Resource limits must be positive integers.

Positive certificate verification

When elimination finds a solution, the solver independently reconstructs the selected component placements and verifies

$$[C] \oplus \bigoplus_j [R_j] = 0$$

by exact physical-edge parity before returning `DECOMPOSITION_FOUND`.

Result boundary

```

@dataclass(frozen=True)
class FiniteRingCancellationResult:
    topology_graph_digest: str
    target_placement: RingPlacement
    candidate_placements: tuple[RingPlacement, ...]
    status: FiniteRingCancellationStatus
    witness: RingCancellationWitness | None

```

This is an in-memory finite-solve record, not the persistent scientific strength schema. The complete candidate tuple is useful for immediate solver diagnostics but is omitted from `RingStrengthResult`.

Complexity

The dense bitset method is appropriate for bounded small-to-medium systems. Its worst-case storage is driven by both support and provenance bitsets. The explicit resource preflight is therefore part of correctness: a resource failure cannot be misreported as nonmembership.

Explicit non-responsibilities

The module does not:

- enumerate translated placements;
- decide catalog lower-closure;
- define incidence depth;
- classify a ring as globally strong;
- serialize a scientific strength result; or
- construct an oriented integer chain complex.

References

The exact finite solver is standard Gaussian elimination over $\text{GF}(2)$. Its application to physical lifted primitive-ring support and the explicit resource/certificate boundary are `mdstats` design decisions. The strong-ring scientific context is attributed in `ring_strength_spec.md`.